Exam quick reference

Group 15 • Final assessment preparation • 3 November 2026

Rubric: final demo 70%, architecture 30%. Excellent requires a working, consistent service under variable event load, notifications **below 10 seconds**, keeping up with variable HTTP load, and identifying bottlenecks under load. Questions can go to any member.

Current audit, 3 November: zero containers after cleanup; named volumes exist but recovery is unverified. Source unchanged from the October report. Fresh unit tests: 42 passed, ten integration tests deselected.

The architecture in one sentence

Kafka -> ordered partition scheduler -> stateless executor -> Data transaction -> PostgreSQL; Redis caches history, Caddy exposes the API, and a separate notifier drains durable outbox jobs.

The atomic commit joins state, history, jobs, rejected input, and checkpoint. Kafka is acknowledged afterward. Losing the acknowledgement causes safe replay. External HTTP acceptance has its own crash window, so delivery is at least once unless the receiver honors idempotency.

Facts to remember

Fact	Correct value or meaning
Demo input	experiment topic; group15-temperature-demo-v1
Public API	Port 3003; /temperature and /temperature/out-of-range
Local panels	Operational 3004; instructor-style 3009
Default execution	One executor, capacity eight; sixteen historical partitions
Task buffering	256-event default batch; pause at 8,192 queued/in-flight events
Notifications	0.1-second empty poll, concurrency 32, timeout 2 seconds, lease 15 seconds
Sensor average	All allocated sensors for the same measurement ID
Boundaries	Temperature thresholds and history time bounds are inclusive
Alert transitions	Stabilized on entry/re-entry during stabilization; one OutOfRange per carry-out excursion
History	Carry-out only; outside history includes every outside point

The evidence story

The final local instructor test used **100 experiments × 10 sensors × 1 Hz**, sixty carry-out samples and 100 offered HTTP/s. One warmup completed before the main experiments.

Measure	Before state-write pipelining	After
Required / received notifications	632 / 632	610 / 610
Above-ten-second notifications	3	0
Maximum latency	10.716 s	5.641 s

Measure	Before state-write pipelining	After
Achieved HTTP/s	86.848	97.310
Complete history comparisons	202 correct	202 correct

The random simulator explains the changed notification count. The observed improvement is 47.36% lower maximum latency and 12.05% higher HTTP throughput, not a repeated controlled causal estimate.

What changed: fetchmany pipelines adjacent sorted state writes. Every version check must return success or the entire transaction rolls back. The regression test makes the middle of three versions stale.

Earlier replica tests: one executor achieved 98.077 HTTP/s; two/four achieved about 93.4–93.6. More workers competed for shared resources. The earlier 200-HTTP/s and twenty-sensor/100-HTTP/s profiles missed the local throughput criterion and need retesting after the later optimization.

Five answers that protect against overclaiming

1. **Exactly once?** Database effects are deduplicated; external notification acceptance is not exactly once without receiver cooperation.
2. **Scale Data?** Its local metadata-version cache requires shared invalidation or removal before multiple processes are safe.
3. **Green dashboard?** Require real load, correct history, required/received alerts, maximum latency, and throughput; no-data green is insufficient.
4. **All workloads passed?** A stated historical local workload passed. Variable professor workloads and current live health need their own evidence.
5. **All data survived cleanup?** Volume names survive; database recovery and content integrity still need verification.

Diagnose before scaling

High CPU + growing processing and HTTP queues: inspect shared-host contention. Slow commits + database waits: inspect Data/database. Current history + old outbox/retries: inspect notification endpoint. Idle workers + one lagging partition: inspect key skew. Growing disk: preserve data and recover capacity, not more workers.

Input rate is E×S/T. Aggregate rate is E/T. Concurrency is bounded by min(partitions, replicas×capacity). None of these formulas by itself gives measured sustainable throughput.

Before you speak

Answer with **decision -> mechanism -> evidence -> limitation**. Distinguish what exists, what was measured, and what is proposed. Every member should explain the transaction, the performance result, a bottleneck, and the next improvement.

Source and details: rubric, design, results, questions, recovery.